

KRUTH CHATBOT

Personality Detection Logic Manual

การคำนวณ ตรรกะ สมการ และการตรวจจับบุคลิกภาพผ่านบทสนทนา

KRUTH APEX / Di Vi Jitr | May 2026 | Confidential — Technical Reference
สังเคราะห์จาก [intent-router.ts](#) + [cla-scoring.ts](#) + [cla-chatbot.ts](#) + [knowledge-retrieval.ts](#)

1. ภาพรวมระบบตรวจจับผ่าน Chatbot

การตรวจจับบุคลิกภาพผ่าน Chatbot ของ KRUTH ต่างจากแบบทดสอบ (DEMM 487 ข้อ) โดยสิ้นเชิง — แทนที่จะให้ตอบ forced-choice ตามโครงสร้างคงที่ ระบบวิเคราะห์ 'ภาษาที่ใช้ตอบ' ในบทสนทนาธรรมชาติ แล้วให้คะแนนผ่าน Claude เป็น scoring engine

1.1 4 Layer ของระบบตรวจจับ

Layer	Module	หน้าที่	Input → Output
1	intent-router.ts	จำแนกว่าข้อความผู้ใช้ต้องการอะไร	message → Intent (14 ประเภท)
2	cla-questions.ts	เลือกคำถามถัดไปแบบ adaptive	session state → next question
3	cla-scoring.ts	ให้คะแนนคำตอบผ่าน Claude + รวมผล	user_message → 0-3 score × 20 มิติ
4	knowledge-retrieval.ts	ให้คำแนะนำที่ตรงกับ profile ที่ตรวจพบ	Intent + profile → relevant KB

หลักการสำคัญ: *Chatbot* ไม่ 'ถามแบบทดสอบ' แต่ 'สนทนาแล้ววิเคราะห์ทีหลัง' — ผู้ใช้ไม่รู้สีกว่ากำลังถูกประเมิน ต่างจาก *DEMM* ที่ผู้ใช้รู้ตัวว่ากำลังทำแบบทดสอบ

2. Layer 1 — Intent Detection (intent-router.ts)

2.1 โครงสร้าง Intent Type — 14 ประเภท

กลุ่ม	Intent Types	Safety Priority
วิกฤต	crisis	crisis (สูงสุด — ข้าม KB ทั้งหมด)
สุขภาพ/สังคม	wellbeing_checkin, youth_social	high
ความสัมพันธ์	toxic_workplace, relationship_advice, compat_query	low
Esports	esports_role, esports_item_build, esports_team, esports_mental, esports_scouting	none / low (mental)
ทั่วไป	personality_query, history_query, checklist_update, general_advice	none

2.2 ตรรกะการตัดสินใจ — 3 ขั้นตอนลำดับ

```
export async function detectIntent(message, userPlatform = 'D'): Promise<Intent> { // ขั้น 1: Safety check ก่อนเสมอ - ตรวจ crisis keyword ก่อนทุกอย่าง if (KEYWORD_MAP.crisis.some(k => message.includes(k))) { return makeIntent('crisis', 1.0, 'keyword', {}, []); // confidence = 1.0 เสมอ, requires_kb = [] (ไม่ต้อง KB ใดๆ) } // ขั้น 2: Keyword matching - เร็ว ไม่เสีย token for (const [intentType, keywords] of Object.entries(KEYWORD_MAP)) { if (intentType === 'crisis' || intentType === 'general_advice') continue; if (keywords.some(k => message.includes(k))) { const entities = extractEntities(message); return makeIntent(type, 0.85, 'keyword', entities, KB_BY_INTENT[type]); // confidence fixed = 0.85 เมื่อ match ด้วย keyword } } // ขั้น 3: Claude classification - เฉพาะเมื่อ keyword ไม่ match return await claudeClassify(message, userPlatform);}
```

⚠ ลำดับนี้สำคัญมาก: Safety check ต้องมาก่อนเสมอ ไม่มีข้อยกเว้น — แม้ keyword อื่นจะ match ก่อนก็ตาม เพราะ crisis คือ life-safety priority

2.3 Claude Classification — เมื่อ Keyword ไม่พอ

```
async function claudeClassify(message, platform): Promise<Intent> { const response = await anthropic.messages.create({ model: 'claude-sonnet-4-20250514', max_tokens: 200, system: `Classify into: ${validIntents}Extract entities: game_name, game_type, toxic_type, target_person, sport_typeRespond ONLY with JSON: {"intent":"...", "confidence":0.0-1.0, "entities":{"...}}`, messages: [{ role: 'user', content: message }], }); const parsed = JSON.parse(text.match(/{\s*\S*\s*}/)?.[0] || '{}'); // ถ้า parse ไม่ได้ → fallback เป็น general_advice, confidence 0.5}
```

2.4 Entity Extraction — ดึงตัวแปรบริบทจากข้อความ

Entity	ตัวอย่าง Keyword ที่ match	ใช้ทำอะไรต่อ
game_name	'lol','league of legends' → 'LoL'	filter KB esports_champion
game_type	'moba','rov','lol','dota' → 'moba'	filter KB esports_role
toxic_type	'จ่าคิด','คิดไปเอง' → 'gaslighting'	filter KB toxic_pattern
target_person	'หัวหน้า'→boss, 'แฟน'→partner	ปรับ tone คำแนะนำ

2.5 Cache TTL — ควบคุมความถี่ในการดึงข้อมูลซ้ำ

```
const cacheTTL: Record<IntentType, number> = { esports_role: 60, esports_item_build: 60, esports_team: 60, esports_scouting: 120, personality_query: 30, history_query: 0, // ไม่ cache - ต้องข้อมูลสดเสมอ wellbeing_checkin: 0, // ไม่ cache - sensitive ต้องสดเสมอ checklist_update: 0, crisis: 0, toxic_workplace: 15, compat_query: 15, youth_social: 0, relationship_advice: 0, general_advice: 30, esports_mental: 15,};
```

Intent ที่เกี่ยวกับความปลอดภัย/สุขภาพ (crisis, wellbeing_checkin, youth_social) ตั้ง cache = 0 เสมอ เพื่อไม่ให้ระบบตอบจากข้อมูลเก่าในสถานการณ์ที่อาจเปลี่ยนแปลงเร็ว

3. Layer 2 — Adaptive Question Selection (cla-questions.ts)

3.1 หลักการ: ไม่ถามตามลำดับตายตัว

ระบบไม่ถามคำถามตามลำดับ 1→2→3→... แต่เลือกคำถามถัดไปจาก 'lens ที่ยังไม่ครบ' และสุ่มจากกลุ่มคำถามที่เหมาะสมกับ trigger (post_training, post_match, weekly_checkin ฯลฯ)

3.2 4 Lens ที่ต้องครบก่อนสรุปผล

Lens	มิติที่วัด	ตัวอย่างคำถาม
Self	attribution, growth mindset, self-efficacy, self-compassion, accountability	"ครั้งสุดท้ายที่เล่นแล้วแพ้ ความคิดแรกที่ผ่านหัวคืออะไร?"
Team	trust, collaboration, conflict tolerance, role acceptance	"เวลาเพื่อนร่วมทีม Feed มาก คุณมักคิดอะไร?"
Opponent	threat framing, respect, emotional reactivity, curiosity	"เวลาเจอทีมที่เล่นดีกว่าเราชัดเจน ความรู้สึกแรกคืออะไร?"
Game	pressure framing, process focus, adaptability, meta-awareness	"เมื่อแผนทีมใช้ไม่ได้กลางเกม คุณมักทำอะไร?"

3.3 Logic การเลือกคำถามถัดไป

```
export function selectNextQuestion(trigger, completedLenses, askedIds, currentLens) { const lensOrder = ['self','team','opponent','game']; const pendingLenses = lensOrder.filter(l => !completedLenses.includes(l)); if (pendingLenses.length === 0) return null; // ครบทุก lens แล้ว const targetLens = pendingLenses[0]; // ทำที่ lens ตามลำดับ const available = QUESTIONS.filter(q => q.lens === targetLens && q.trigger.fits.includes(trigger) && // ต้องเหมาะกับบริบท (หลังแพ้/หลังอบรม/ ฯลฯ) !askedIds.includes(q.id) // ไม่ถามซ้ำใน session เดียวกัน ); if (available.length === 0) return null; // สุ่มจาก 3 ตัวแรกที่ match - ป้องกันถามคำถามเดิมทุก session return available[Math.floor(Math.random() * Math.min(available.length, 3))];}
```

3.4 เกณฑ์ 'ครบ Lens' — แตกต่างตาม Trigger

```
export function isLensComplete(lens, askedCount, trigger) { const minPerLens = { post_training: 2, // อบรม → ถามลึกกว่า post_match: 1, // หลังเกม → เร็ว ไม่รบกวนนาน post_loss: 2, // หลังแพ้ → สำคัญถามละเอียดขึ้น post_win: 1, weekly_checkin: 2, pre_tournament: 1, on_request: 1, }; return askedCount >= (minPerLens[trigger] ?? 2);}
```

Trigger 'post_loss' และ 'post_training' ต้องการคำถาม 2 ข้อต่อ lens (ลึกกว่า) เพราะเป็นช่วงเวลาที่ signal ทางจิตวิทยาชัดเจนที่สุด — 'post_match' ทั่วไปใช้แค่ 1 ข้อเพื่อไม่ให้อึดอัดถามซ้ำหลังทุกเกม

4. Layer 3 — Scoring Engine (cla-scoring.ts)

4.1 หลักการ: Claude เป็น Scoring Engine ไม่ใช่แค่ Chat

ทุกคำตอบของผู้ใช้ถูกส่งให้ Claude วิเคราะห์แยกต่างหาก (ไม่ใช่ Claude ตัวเดียวกับที่คุยอยู่) โดยมี System Prompt เฉพาะสำหรับการให้คะแนนเท่านั้น แยกจาก Prompt การสนทนา

4.2 มาตรฐานคะแนน — 0-3 ต่อมิติ (20 มิติทั้งหมด)

ค่า	ความหมาย	ตัวอย่าง Self-Lens	ตัวอย่าง Opponent-Lens
0	แง่ลบชัดเจน	attribution_locus: โทษภายนอกทั้งหมด	threat_framing: มองคู่ต่อสู้เป็นภัย
1	แนวโน้มลบ/ไม่ชัดเจน	—	—
2	แนวโน้มบวก/กลางๆ	—	—
3	แง่บวกชัดเจน	attribution_locus: รับผิดชอบตัวเองเต็มที่	threat_framing: มองเป็น challenge

4.3 ครบ 20 มิติ แบ่ง 4 กลุ่ม

กลุ่ม	มิติ (5 ต่อกลุ่ม)
Self-Lens	attribution_locus, growth_mindset, self_efficacy, self_compassion, accountability
Team-Lens	team_trust, collaboration_will, conflict_tolerance, role_acceptance, contribution_aware
Opponent-Lens	threat_framing, opponent_respect, emotional_reactivity, analytical_curiosity, sportsmanship
Game-Lens	pressure_framing, process_vs_outcome, adaptability_mind, meta_awareness, long_term_thinking

4.4 Prompt Structure ที่ส่งให้ Claude ให้คะแนน

```
const prompt = `คำถาม: "${questionTh}" คำตอบผู้เล่น: "${userMessage}" บท: Quad=${context.quad} ผลเกม=${context.game_result} มิติที่วัด: ${dimensionsMeasured.join(', ')} ให้: 1. scores: {[dimension]: 0-3} เฉพาะมิติที่วัดเท่านั้น 2. signals: [{signal_type, polarity, keywords, quote, dimension}] 3. confidence: 0.0-1.0 JSON`;
```

4.5 Language Signal — เก็บหลักฐานจากคำพูดจริง

ทุกครั้งที่ให้คะแนน ระบบไม่ได้เก็บแค่ตัวเลข แต่เก็บ 'สัญญาณภาษา' (Language Signal) ที่เป็นหลักฐานประกอบคะแนนนั้นด้วย

Field	ค่าที่เป็นไปได้	ตัวอย่าง
signal_type	attribution / mindset / emotion / framing / social	"ทีมห่วย" → attribution
polarity	positive / negative / neutral	"ทีมห่วย" → negative
quote	ข้อความที่ตัดมาจากคำตอบจริง	เก็บคำต้นฉบับเพื่อ traceability

4.6 Aggregation — รวมคะแนนจากหลายคำตอบเป็น Profile เดียว

```
export function aggregateScores(responses: CLAResponse[]) { const allScores: Record<string, number[]> = {}; // รวมรวมคะแนนทุกมิติจากทุกคำตอบใน session for (const r of responses) { for (const [dim, score] of Object.entries(r.scores)) { allScores[dim].push(score); } } //เฉลี่ยแต่ละมิติ - ถ้าไม่มีข้อมูล ใช้ 1.5 (neutral default) for (const key of dimKeys) { dims[key] = scores.length > 0 ? Math.round((scores.reduce((a,b)=>a+b,0) / scores.length) * 10) / 10 : 1.5; } //เฉลี่ยรวมเป็น Lens Score (5 มิติ → 1 คะแนน lens) lens.self_lens = avg(selfDims); lens.team_lens = avg(teamDims); // ... lens.overall = avg(4 lens scores); return { dims, lens }; }
```

ค่า *default* 1.5 (จุดกึ่งกลางของ *scale* 0-3) ถูกใช้เมื่อมิตินั้นยังไม่มีข้อมูล — ป้องกันไม่ให้มิตินี้ยังไม่ถูกถามแสดงเป็น 0 (แง่ลบ) อย่างผิดพลาด

5. DEMM Deviation — เทียบผลจาก Chatbot กับ Baseline

5.1 หลักการ: ตรวจสอบพฤติกรรมตรงกับ Quad ที่ DEMM วัดไว้ไหม

ระบบมี Expected Score ที่คาดหวังสำหรับแต่ละ Quadrant (จาก DEMM) แล้วเทียบกับคะแนนจริงที่ได้จากการสนทนา — ถ้าต่างกันมาก (deviation) แปลว่ามีสิ่งที่น่าสนใจเกิดขึ้น ไม่ตรงกับ pattern ปกติของคนคนนั้น

5.2 Expected Score ตาม Quadrant

Quad	มิติที่คาดหวังสูง (Expected)
Q1	threat_framing: 2.5 adaptability_mind: 2.5 pressure_framing: 2.5 team_trust: 2.0
Q2	analytical_curiosity: 2.8 process_vs_outcome: 2.5 meta_awareness: 2.5 long_term_thinking: 2.5
Q3	team_trust: 2.8 collaboration_will: 2.8 conflict_tolerance: 2.5 opponent_respect: 2.5
Q4	accountability: 2.5 process_vs_outcome: 2.0 role_acceptance: 2.5 long_term_thinking: 2.3

5.3 สูตรคำนวณ Deviation

```
export function calcDeviations(dims: CLADimensionScores, quad?: string) { if (!quad || !OCEAN_TO_CLA_EXPECTED[quad]) return []; const deviations = []; for (const [dim, expected] of Object.entries(OCEAN_TO_CLA_EXPECTED[quad])) { const actual = dims[dim] ?? 1.5; const gap = actual - expected; if (Math.abs(gap) >= 0.5) { // threshold: ต่างกัน ≥ 0.5 ถึงจะนับ deviations.push({ dimension: dim, expected, actual, gap, meaning: gap > 0 ? "...สูงกว่าที่คาด" : "...ต่ำกว่าที่คาด" }); } } return deviations.sort((a,b) => Math.abs(b.gap) - Math.abs(a.gap)); // เรียงจาก deviation ที่มากที่สุดก่อน }
```

Threshold ที่ 0.5 หมายความว่า deviation เล็กน้อย (< 0.5) จะไม่ถูกนับเป็นสัญญาณที่มีนัยสำคัญ — ป้องกัน noise จากความคลาดเคลื่อนปกติของการวัดครั้งเดียว

6. Pattern Detection — จำแนกรูปแบบพฤติกรรม 8 แบบ

6.1 หลักการ: Rule-based Pattern จากคู่มิติ

หลังได้คะแนนครบ 20 มิติ ระบบตรวจสอบว่าคะแนนตรงกับ 'Pattern' ที่กำหนดไว้ล่วงหน้าไหม — แต่ละ Pattern ใช้เงื่อนไข 2 มิติร่วมกัน (ไม่ใช่มิติเดียว) เพื่อลดโอกาสจำแนกผิดจากสัญญาณเดียว

6.2 ตาราง Pattern ทั้ง 8 แบบ

ID	Pattern	เงื่อนไข (check function)	Evidence Dims
P001	Blamer (มักโทษคนอื่น)	attribution_locus $\leq$ 1.0 AND accountability $\leq$ 1.5	attribution_locus, accountability
P002	Growth Seeker	growth_mindset $\geq$ 2.5 AND process_vs_outcome $\geq$ 2.0	growth_mindset, process_vs_outcome
P003	Team Anchor (หลักของทีม)	team_trust $\geq$ 2.5 AND collaboration_will $\geq$ 2.5	team_trust, collaboration_will
P004	Lone Wolf (นักรบเดี่ยว)	collaboration_will $\leq$ 1.5 AND team_trust $\leq$ 1.5	collaboration_will, team_trust
P005	Pressure Player	pressure_framing $\geq$ 2.5 AND meta_awareness $\geq$ 2.0	pressure_framing, meta_awareness
P006	Reactive Fighter	emotional_reactivity $\leq$ 1.5 AND threat_framing $\leq$ 1.5	emotional_reactivity, threat_framing
P007	Strategic Thinker	analytical_curiosity $\geq$ 2.5 AND long_term_thinking $\geq$ 2.5	analytical_curiosity, long_term_thinking
P008	Mental Fragile (ใจเปราะบาง)	pressure_framing $\leq$ 1.0 AND emotional_reactivity $\leq$ 1.0	pressure_framing, emotional_reactivity

6.3 Strength Rating — ความมั่นใจของ Pattern ที่ตรวจพบ

```
const strength = pat.evidence_dims.every(d => Math.abs(dims[d] - 1.5) >= 1.0) // ทุกมิติต้องห่างจากจุดกึ่งกลางมาก)
? 'strong' : 'moderate'; // ตัวอย่าง: Blamer pattern// attribution_locus = 0.3 (ห่างจาก 1.5 = 1.2 →  $\geq 1.0$  ✓)//
accountability = 0.8 (ห่างจาก 1.5 = 0.7 →  $< 1.0$  ✗) // → strength = 'moderate' (ไม่ใช่ 'strong' เพราะมิติที่ 2 ยังไม่ห่างพอ)
```


7. Coaching Tips Generator — จาก Pattern สู่คำแนะนำ

7.1 หลักการ: Threshold-based Tip Selection

แต่ละมิติมี threshold ที่ trigger คำแนะนำเฉพาะ — ระบบไม่ generate คำแนะนำจาก Claude สดๆ แต่เลือกจาก Tip Bank ที่เตรียมไว้ล่วงหน้า (คล้ายหลักการ pre-scripted response ที่ใช้กับ AI Character สำหรับเด็ก)

7.2 ตัวอย่าง Trigger Condition → Tip

Lens	Trigger	Tip (ย่อ)	Theory Link
Self	attribution_locus < 1.5	Attribution Retraining — เขียน 'สิ่งที่ทำได้ดีกว่านี้' ก่อนพูดถึงทีม/ดวง	CB_001 (Cognitive Reframing)
Team	team_trust < 1.5	Small Wins — สร้าง reliability ในทีมทีละขั้น	AT_001 (Attachment & Trust)
Opponent	threat_framing < 1.5	Reframe ก่อนแข่ง: 'พวกเขาแข็งแกร่งและฉันเรียนรู้ได้'	CB_001 (Cognitive Appraisal)
Game	pressure_framing < 1.5	Pre-clutch ritual: หายใจ 4-4-4 + self-talk	EI_001 + AC_001

7.3 Priority Sorting — เลือกคำแนะนำสำคัญสุดก่อน

```
return tips.sort((a, b) => a.priority === 'high' ? -1 : b.priority === 'high' ? 1 : 0).slice(0, 5); // จำกัด  
สูงสุด 5 tips ต่อรอบ - ไม่ให้ผู้ใช้ overload
```

8. Layer — Conversation Workflow (cla-chatbot.ts)

8.1 Safety Gate — ตรวจสอบทุก Message

```
const crisisKeywords = ['อยากตาย', 'ฆ่าตัวตาย', 'ทนไม่ไหว', 'สิ้นหวัง', 'หมดหวังแล้ว']; if (crisisKeywords.some(k => userMessage.includes(k))) { return { botReply: "...สายด่วนสุขภาพจิต 1323...", session, isComplete: false }; } // หยุดกระบวนการ scoring ทั้งหมดทันที ไม่ประมวลผลต่อ
```

8.2 Adaptive Follow-up — ถามซ้ำเมื่อคำตอบไม่ชัด

```
const shouldFollowUp = lastResponse && lastResponse.confidence < 0.6 // confidence ต่ำจาก scoring engine && !lastResponse.follow_up_used; // follow-up ใช้ได้แค่ 1 ครั้งต่อคำถาม const followUpQ = shouldFollowUp && lastQ?.sub_question ? lastQ.sub_question : null; // ถ้า follow-up: ถามคำถามเสริมที่เตรียมไว้ล่วงหน้า (ไม่ generate สด) // ถ้าไม่: Acknowledge สั้นๆ แล้ว transition ไปคำถามถัดไปอย่างธรรมชาติ
```

confidence < 0.6 คือ *threshold* ที่กำหนดให้ระบบ “ไม่มั่นใจพอ” ในคำตอบที่ *scoring engine* วิเคราะห์ได้ — เกิดขึ้นเมื่อคำตอบผู้ใช้กำกวมหรือสั้นเกินไปจนตีความยาก

8.3 Session Completion — เมื่อครบทุก Lens

เมื่อ 4 Lens ครบตามเกณฑ์ (isLensComplete) ระบบเรียก buildCLAProfile() ซึ่งรวม 3 ฟังก์ชันจาก Layer 3 เข้าด้วยกัน:

```
function buildCLAProfile(session: CLASession): CLAProfile { const { dims, lens } = aggregateScores(session.responses); const deviations = calcDeviations(dims, session.match_context?.role); const patterns = detectPatterns(dims); const tips = generateCoachingTips(lens, dims); return { user_id, session_id, lens_scores: lens, dim_scores: dims, demm_deviation: deviations, patterns, coaching_tips: tips, ... }; }
```

9. Layer 4 — Relevance Scoring สำหรับดึงคำแนะนำ (knowledge-retrieval.ts)

9.1 หลักการ: ใช้ Profile ที่ตรวจพบ กรอง Knowledge Base

เมื่อรู้ Quad/Jung/VIA ของผู้ใช้แล้ว (จาก DEMM หรือจาก CLA pattern) ระบบใช้ค่าเหล่านี้เป็นตัวกรองเพื่อเลือกเนื้อหาที่ตรงจุดที่สุดจาก Knowledge Base แทนที่จะส่งเนื้อหาทั้งหมดให้ Claude

9.2 สูตร Relevance Score (Structured Lookup)

```
function calcStructuredRelevance(row, ctx, entities): number { let score = 0.5; // baseline const meta = row.metadata || {}; if (meta.quad_primary === ctx.quad) score += 0.3; // ตรง Quad หลัก if (meta.quad_secondary === ctx.quad) score += 0.15; // ตรง Quad รอง if (meta.jung_ideal?.includes(ctx.jung)) score += 0.2; // ตรง Jungian if (meta.via_ideal?.includes(ctx.via)) score += 0.1; // ตรง VIA if (entities.game_name && meta.game === entities.game_name) score += 0.2; if (entities.game_type && meta.game_type?.includes(entities.game_type)) score += 0.15; return Math.min(1.0, score); // cap ที่ 1.0 }
```

เงื่อนไขที่ตรง	น้ำหนัก	เหตุผล
Quad หลักตรง	+0.30	น้ำหนักสูงสุด — Quad คือมิติหลักของ Compat/Role
Jungian ตรง	+0.20	รองลงมา — เสริมความแม่นยำ
Game name ตรง	+0.20	สำคัญเมื่อ context เป็น esports เฉพาะเกม
Quad รองตรง	+0.15	borderline quad ยังนับ แต่น้อยกว่า
Game type ตรง	+0.15	กว้างกว่า game name (เช่น moba ทั้งหมด)
VIA ตรง	+0.10	น้ำหนักต่ำสุด — เสริมเท่านั้น

10. สรุปภาพรวม Pipeline ทั้งหมด

```
ผู้ใช้พิมพ์ข้อความ | ▼[Layer 1] detectIntent() Safety keyword? —Yes—> Crisis response ทันที (จบ pipeline) | No Keyword match? —Yes—> Intent (confidence 0.85, fast) | No Claude classify  
—> Intent (confidence จาก Claude) | ▼[Layer 2] selectNextQuestion() หา lens ที่ยังไม่ครบ ->  
ถามคำถามที่เหมาะสมกับ trigger | ผู้ใช้ตอบคำถาม (ภาษาธรรมชาติ) | ▼[Layer 3]  
scoreResponse() — Claude scoring engine -> {scores: 0-3 ต่อมิติ, signals: [], confidence: 0-1}  
confidence < 0.6? —Yes—> ถาม follow-up (ครั้งเดียว) | No ครบทุก lens? —No—> วนกลับไป  
selectNextQuestion() | Yes ▼ buildCLAProfile() | aggregateScores() -> dims (20) + lens  
(4) | calcDeviations() -> เปรียบกับ DEMM baseline | detectPatterns() -> จับคู่ 8 Pattern |  
generateCoachingTips() -> เลือก tip จาก threshold | ▼[Layer 4] retrieveKnowledge() ใช้  
Quad/Jung/VIA ที่ได้ -> คำนวณ Relevance Score -> ดึงเนื้อหาที่ตรงที่สุดจาก KB | Claude สร้างคำตอบ  
สุดท้ายให้ผู้ใช้
```

10.1 หลักการที่ยึดตลอด Pipeline

- Safety check มาก่อนเสมอ ทุก Layer — ไม่มีข้อยกเว้น
- Scoring engine (Claude) แยกจาก Conversation engine (Claude) — คนละ system prompt คนละหน้าที่
- ทุกคะแนนมี default (1.5) ป้องกันมิติที่ยังไม่ถูกถามแสดงผลผิดเป็นแง่ลบ
- Pattern ต้องใช้ 2 มิติร่วมกันเสมอ ไม่ตัดสินจากมิติเดียว
- Deviation จาก DEMM baseline ใช้ threshold 0.5 กันสัญญาณ noise
- Follow-up question ใช้ได้แค่ 1 ครั้งต่อคำถาม ป้องกันสัมภาษณ์เยื้อเยื้อเกินไป